MBLP131 · Hafta 10 · Ders Notları

İşletim Sistemleri Temelleri

Dosya ve Dizin İşlemleri

Dosya oluşturun, kopyalayın, taşıyın, silin, görüntüleyin, bağlayın — ve her seferinde çekirdeğin ne yaptığını bilin.

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. `touch`, `cp`, `mv`, `ln`, `ln -s` ve `rm` komutlarının neyi değiştirdiğini dizin girdileri, düğümler (inode) ve veri blokları (7. hafta) cinsinden açıklamak.
2. `mkdir` (`-p`) ve `touch` ile dizin ve dosya oluşturmak; atime, mtime ve ctime zaman damgalarını okumak.
3. `cp`, `mv`, `rm`, `rmdir` ile dosyaları ve dizin ağaçlarını kopyalamak, taşımak, yeniden adlandırmak ve silmek — güvenli biçimde.
4. `*`, `?`, `[ ]` joker karakterlerini ve `{a,b}`, `{1..5}` süslü parantez genişletmesini kullanmak; bunları *kabuğun* genişlettiğini açıklamak.
5. Adında boşluk bulunan dosyalarla çalışmak.
6. Dosyaları `cat`, `less`, `head`, `tail`, `wc`, `file`, `stat`, `du` ve `df` ile incelemek.
7. Sabit ve sembolik bağlantılar oluşturmak ve özgün dosya silinince ne olacağını önceden söylemek.

İçindekiler

1 9. Haftadan 10. Haftaya 1

2 Adlar, Düğümler ve Sistem Çağrıları 2

3 Oluşturmak: `mkdir` ve `touch` 3

4 Kopyalamak, Taşımak ve Silmek 5

5 Joker Karakterler ve Süslü Parantez Genişletmesi 8

6 Boşluklu Adlar: Tırnaklama 10

7 Dosyaların İçine Bakmak 11

8 Bağlantılara Yeniden Bakış 13

9 Komut Başvuru Tablosu 16

10 Sık Görülen Hatalar ve Anlamları 16

11 Özet 17

12 Sözlük 18

13 Kendiniz Deneyin 18

14 Tekrar Soruları 18

15 Laboratuvar Föyü (notlu, 100 puan) 19

16 Laboratuvar Föyü Çözümleri 20

17 İleriye Bakış: 11. Hafta 23

1 9. Haftadan 10. Haftaya

9. haftada terminal ve kabukla (shell) tanıştık, bir komutun yapısını öğrendik (`komut -seçenekler argümanlar`) ve Linux dizin ağacında `pwd`, `ls` ve `cd` ile dolaştık; bunu yaparken mutlak yollar (`/home/student/lab`) ve göreli yollar (`docs`, `../data`) kullandık. Yalnızca *baktık*. Bu hafta dosya sistemini *değiştiriyoruz*: dosya ve dizin oluşturacak, kopyalayacak, taşıyacak, yeniden adlandıracak, silecek ve bağlayacak; ayrıca dosyaların içine bakacağız.

Buradaki her şey 7. haftanın dosya sistemi kuramına dayanır; bu yüzden kısa bir tekrarla başlıyoruz. Ardından komutlar geliyor; her biri gerçek çıktısıyla gösteriliyor.

Hazırlık

Bütün örnekler laboratuvar betiğinin oluşturduğu alıştırma klasöründe çalışır (betik her Linux haftasının `code/` klasöründe bulunur). Her dersten önce çalıştırın; betik *yalnızca* `~/lab` klasörünü silip yeniden oluşturur:

```
$ bash ~/mcmp117_lab_setup.sh
$ cd ~/lab
```

Slaytlardaki her gösterim `code/` içinde bir betiktir (örneğin `demo_cp_mv.sh`); yakalanan çıktısı da eşleşen `out_*.txt` dosyasıdır. `student@okan-lab:~/lab$` istemi (prompt) kullanıcıyı, makineyi ve geçerli dizini gösterir; siz yalnızca `$` işaretinden sonrasını yazarsınız.

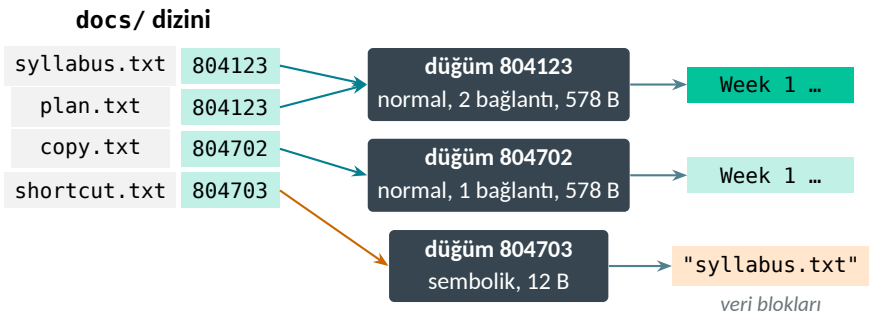
Sizin çıktınız biraz farklı olacak

Çıktılar sıradan kullanıcı `student` ile ve `LC_ALL =C.UTF-8` ayarıyla yakalandı. Başka yerel ayarlarda (locale) `ls` adları farklı *sıralayabilir* (C yerel ayarında büyük harfler önce gelir, bu yüzden `IMG_014.JPG`, `holiday.jpeg`'den önce listelenir). Türkçe yerel ayarlı bir sistemde (`LANG =tr_TR.UTF-8`) pek çok ileti Türkçe görünür ve sıralama yine farklı olabilir; ders çıktıları bu nedenle `LC_ALL =C.UTF-8` ile alınmıştır. Tarihler, saatler (bizimkiler UTC, `+0000`; Türkiye'de `+0300` göreceksiniz), düğüm numaraları, disk boyutları ve kullanıcı kimlikleri sizin makinenizde farklı olacaktır. Geri kalan her şey aynı olmalıdır.

2 Adlar, Düğümler ve Sistem Çağrıları

2.1 Tekrar: ne nerede saklanır

7. haftadan hatırlayın: bir **dizin**, (ad, düğüm numarası) çiftlerinden oluşan bir tablodur. **Düğüm** (inode) bir dosyayla ilgili geri kalan her şeyi tutar — tür, boyut, sahip, izinler, zaman damgaları, **bağlantı sayısı** (link count) ve **veri bloklarına** (data blocks) işaretçiler — ama dosyanın adını *tutmaz*. Bir düğümün birden çok adı olabilmesinin ve bazı işlemler anında biterken bazılarının veri kopyalamasının nedeni budur.



Şekil 1: Bağlantı gösterimindeki gerçek durum (Kısım 8): `plan.txt` bir sabit bağlantı, `copy.txt` bir kopya, `shortcut.txt` bir sembolik bağlantıdır.

Bu resimle haftanın her komutu *neyi değiştirdiğiyle* tanımlanabilir:

Komut	Ne değişir	Sistem çağrısı
`touch new`	yeni bir ad ve yeni (boş) bir düğüm	`openat(O_CREAT)`
`mkdir d`	yeni bir ad ve yeni bir dizin düğümü	`mkdir()`
`cp a b`	yeni bir ad, yeni bir düğüm *ve* bütün verinin kopyası	`open/read/write`
`mv a b`	yalnızca dizin girdileri; düğüm aynı, veriye dokunulmaz	`renameat2()`
`ln a b`	*aynı* düğümü gösteren yeni bir ad; bağlantı sayısı $+1$	`linkat()`
`ln -s a b`	yeni bir ad ve verisi a yolu olan küçük bir düğüm	`symlinkat()`
`rm a`	bir ad silinir; bağlantı sayısı -1; 0 olunca veri serbest kalır	`unlinkat()`
`rmdir d`	boş bir dizin silinir	`rmdir()`

2.2 Komutlar programdır; işlerini sistem çağrıları yapar

Her komut `/usr/bin` içindeki küçük bir programdır. Kabuk onu `fork()` ve `exec()` ile başlatır (3. hafta); program da asıl işi bir **sistem çağrısı** (system call) aracılığıyla çekirdekten ister (2. hafta). Bunu `strace` ile izleyebiliriz; `-e trace =…` yalnızca adını verdiğimiz çağrıları gösterir (betik `demo_strace.sh`; her komuttan sonraki `+++ exited with 0 +++` satırları çıkarıldı):

```
student@okan-lab:~/lab$ strace -e trace=mkdir,mkdirat mkdir newdir
mkdir("newdir", 0777)                   = 0
student@okan-lab:~/lab$ strace -e trace=rename,renameat,renameat2 mv newdir olddir
renameat2(AT_FDCWD, "newdir", AT_FDCWD, "olddir", RENAME_NOREPLACE) = 0
student@okan-lab:~/lab$ touch file1
student@okan-lab:~/lab$ strace -e trace=link,linkat ln file1 file2
linkat(AT_FDCWD, "file1", AT_FDCWD, "file2", 0) = 0
student@okan-lab:~/lab$ strace -e trace=symlink,symlinkat ln -s olddir sym
symlinkat("olddir", AT_FDCWD, "sym")    = 0
student@okan-lab:~/lab$ strace -e trace=unlink,unlinkat rm file1 file2 sym
unlinkat(AT_FDCWD, "file1", 0)          = 0
unlinkat(AT_FDCWD, "file2", 0)          = 0
unlinkat(AT_FDCWD, "sym", 0)            = 0
student@okan-lab:~/lab$ strace -e trace=rmdir,unlinkat rmdir olddir
rmdir("olddir")                         = 0
```

Dikkat edin:

- `mv` tek bir bayt bile kopyalamadı: yalnızca bir `renameat2()` çağrısı yaptı. `RENAME_NOREPLACE` bayrağı “hedef varsa başarısız ol” demektir; `mv` önce bunu kullanır ve bir hedefin yerine ancak bilerek yazar.
- Üç adlı `rm`, üç `unlinkat()` çağrısı yapar. Çekirdek silme işlemine *unlink* der: bir düğümün tek bir *bağlantısını* (adını) kaldırır.
- `…at` sürümleri ilk argüman olarak bir dizin alır; `AT_FDCWD` “geçerli çalışma dizinine göre” demektir — 9. haftadaki göreli yolların ta kendisi.

3 Oluşturmak: `mkdir` ve `touch`

3.1 `mkdir`: dizin oluşturmak

`mkdir ad` boş bir dizin oluşturur. İki hata sık görülür: ad zaten kullanılıyordur ya da yol üzerindeki bir *üst* dizin yoktur. `-p` seçeneği (*parents*, üst dizinler) yol üzerindeki eksik bütün dizinleri oluşturur ve dizin zaten varsa hiçbir şey söylemez (`demo_mkdir_touch.sh`):

```
student@okan-lab:~/lab$ mkdir test
student@okan-lab:~/lab$ ls -ld test
drwxrwxr-x 2 student student 4096 Oct 12 08:47 test
student@okan-lab:~/lab$ mkdir test
mkdir: cannot create directory ‘test’: File exists
student@okan-lab:~/lab$ mkdir trip/day1
mkdir: cannot create directory ‘trip/day1’: No such file or directory
student@okan-lab:~/lab$ mkdir -p trip/day1
student@okan-lab:~/lab$ mkdir -p work/2026/october
student@okan-lab:~/lab$ tree work trip
work
└── 2026
```

```
    └── october
trip
└── day1

5 directories, 0 files
```

`ls -ld test`, içeriği değil dizinin *kendisini* listeler (`-d`). Bağlantı sayısı 2'dir: `~/lab` içindeki `test` adı ve dizinin içindeki `.` girdisi. `tree` bir dizin ağacı çizer; sondaki sayım başlangıç dizinlerini de içerir (5: `work`, `2026`, `october`, `trip`, `day1`).

3.2 `touch`: boş dosyalar ve zaman damgaları

`touch dosya` komutunun iki etkisi vardır. Dosya yoksa 0 boyutlu olarak oluşturulur. Varsa içeriğine dokunulmaz, yalnızca zaman damgaları "şimdi"ye ayarlanır. `-d` ile istediğiniz bir zamanı ayarlayabilirsiniz:

```
student@okan-lab:~/lab$ touch todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Oct 12 08:47 todo.txt
student@okan-lab:~/lab$ touch -d "2026-01-15 09:30" todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Jan 15  2026 todo.txt
student@okan-lab:~/lab$ stat todo.txt
  File: todo.txt
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 254,0   Inode: 804708      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-01-15 09:30:00.000000000 +0000
Modify: 2026-01-15 09:30:00.000000000 +0000
Change: 2026-10-12 08:47:56.442757013 +0000
 Birth: 2026-10-12 08:47:56.430757013 +0000
student@okan-lab:~/lab$ touch todo.txt
student@okan-lab:~/lab$ ls -l todo.txt
-rw-rw-r-- 1 student student 0 Oct 12 08:47 todo.txt
```

`ls -l` komutunun `Jan 15  2026` gösterdiğine dikkat edin — günün saati yerine yıl — çünkü tarih bugünden altı aydan daha uzaktır. `stat` (Kısım 7.4) düğümde tutulan dört zamanın hepsini gösterir.

3.3 Üç (ya da dört) zaman damgası

Bir düğüm üç zaman tutar: **atime** (Access, verinin son okunması), **mtime** (Modify, verinin son değişmesi — `ls -l` komutunun gösterdiği zaman) ve **ctime** (Change, *düğümün* son değişmesi: veri, bağlantı sayısı, izinler, sahip, zamanlar). **ctime oluşturulma zamanı değildir**; ext4 gibi daha yeni dosya sistemleri ayrıca bir oluşturulma zamanı da kaydeder; `stat` bunu *Birth* olarak gösterir.

Hangi işlemin hangi zamanı değiştirdiğini test makinemizde (ext4) doğruladık:

İşlem	atime	mtime	ctime
okuma: `cat f`	evet (gecikmeli)	—	—
yazma: `echo x >> f`	—	evet	evet
`touch f`	evet	evet	evet
yeniden adlandırma: `mv f g`	—	—	evet
yeni sabit bağlantı: `ln f h`	—	—	evet
izinler: `chmod` (11. hafta)	—	—	evet

"Gecikmeli": her okumada diske yazmamak için Linux dosya sistemlerini `relatime` seçeneğiyle bağlar ve atime'ı yalnızca mtime'dan eskiyse ya da 24 saatten eskiyse günceller. Yedekleme programları ve `make` mtime'a güvenir; `cp -p`, kopyaya o anki zamanı vermek yerine dosyanın özgün mtime'ını korur.

4 Kopyalamak, Taşımak ve Silmek

4.1 cp: kopyalamak

`cp KAYNAK HEDEF` bir dosyayı kopyalar. `HEDEF` var olan bir dizinse kopya adını korur ve onun içine gider; değilse `HEDEF` yeni addır. Birden çok kaynak varsa son argüman bir dizin olmalıdır. Seçenekler: `-v` (verbose, ayrıntılı) her kopyalamayı bildirir, `-i` (interactive, etkileşimli) üzerine yazmadan önce sorar, `-r` (recursive, özyinelemeli) dizinleri kopyalar (`demo_cp_mv.sh`):

```
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/syllabus_copy.txt
student@okan-lab:~/lab$ ls backup
syllabus.txt  syllabus_copy.txt
student@okan-lab:~/lab$ cp -v docs/notes_01.txt docs/notes_02.txt backup/
'docs/notes_01.txt' -> 'backup/notes_01.txt'
'docs/notes_02.txt' -> 'backup/notes_02.txt'
student@okan-lab:~/lab$ cp -i docs/syllabus.txt backup/
cp: overwrite 'backup/syllabus.txt'? n
student@okan-lab:~/lab$ cp projects/alpha backup/
cp: -r not specified; omitting directory 'projects/alpha'
student@okan-lab:~/lab$ cp -r projects/alpha backup/
student@okan-lab:~/lab$ tree backup
backup
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
├── notes_01.txt
├── notes_02.txt
├── syllabus.txt
└── syllabus_copy.txt

4 directories, 7 files
student@okan-lab:~/lab$ ls -i docs/syllabus.txt backup/syllabus.txt
804330 backup/syllabus.txt  804123 docs/syllabus.txt
```

Gözlemler:

- `cp -i` üzerine yazmadan önce sordu; `n` (hayır) yanıtını verdik. (Betikte yanıtı `echo n` sağlar; terminalde siz yazarsınız.) `-i` **olmadan** `cp` var olan bir dosyanın üzerine sessizce yazar.
- Bir dizin `-r` olmadan kopyalanmaz. *Özyinelemeli* şu demektir: dizini kopyala, sonra içindeki her girdi için aynısını yap ve ağaç boyunca böyle aşağı in.
- `ls -i` *farklı* düğüm numaraları gösterir (804330 ve 804123): kopya yeni ve bağımsız bir dosyadır. Birini değiştirmek öbürünü değiştirmez.

Yararlı ekler: `cp -p` zamanları ve izinleri de korur; `cp -a` (archive, arşiv) `-r` artı `-p`'nin koruduğu her şeydir ve yedekleme için istediğiniz budur.

> **Sessiz bir tuzak**
>
> `backup/` *yoksa* `cp notes.txt backup` komutu `backup` adında bir *dosya* oluşturur. Sona eğik çizgi koyarak `backup/` yazarsanız `cp` bunun yerine hata vererek durur.

4.2 `mv`: yeniden adlandırmak ve taşımak

Linux'ta tek dosyalar için ayrı bir yeniden adlandırma komutu yoktur: `mv` ikisini de yapar. Aynı dizin içinde `mv eski yeni` yeniden adlandırır; `mv dosya dizin/` adı başka bir dizine taşır. Dizinler de aynı biçimde, `-r` olmadan taşınır:

```
student@okan-lab:~/lab$ mv backup/syllabus.txt backup/syllabus_old.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_old.txt
804330 backup/syllabus_old.txt
student@okan-lab:~/lab$ mv -v docs/notes_03.txt backup/
renamed 'docs/notes_03.txt' -> 'backup/notes_03.txt'
student@okan-lab:~/lab$ mv backup/alpha backup/alpha_2026
student@okan-lab:~/lab$ ls docs backup
backup:
alpha_2026    notes_02.txt  syllabus_copy.txt
notes_01.txt  notes_03.txt  syllabus_old.txt

docs:
'final report.txt'   notes_01.txt   syllabus.txt
 linux_quotes.txt    notes_02.txt
```

`backup/syllabus.txt` dosyasının düğüm numarası yeniden adlandırmadan önce 804330 idi ve sonra da 804330'dur: yalnızca dizin girdisi değişti (tek bir `renameat2()` çağrısı, Kısım 2). Bu yüzden aynı dosya sistemi içinde `mv`, dosya boyutu ne olursa olsun anında biter. İki dosya sistemi arasında (örneğin diskten bir USB belleğe) düğüm taşınamaz; bu durumda `mv` veriyi kopyalar, sonra özgün dosyayı siler. `cp` gibi `mv` de var olan bir hedefin üzerine sessizce yazar; `mv -i` kullanın.

4.3 `rm` ve `rmdir`: silmek

`rm` adları siler; `rmdir` *boş* dizinleri siler (`demo_rm.sh`):

```
student@okan-lab:~/lab$ touch junk1.txt junk2.txt junk3.txt
student@okan-lab:~/lab$ rm junk1.txt
student@okan-lab:~/lab$ rm -i junk2.txt
rm: remove regular empty file 'junk2.txt'? y
```

```
student@okan-lab:~/lab$ rm -v junk3.txt
removed 'junk3.txt'
student@okan-lab:~/lab$ ls junk*
ls: cannot access 'junk*': No such file or directory
student@okan-lab:~/lab$ mkdir empty_dir
student@okan-lab:~/lab$ rmdir empty_dir
student@okan-lab:~/lab$ rmdir projects/beta
rmdir: failed to remove 'projects/beta': Directory not empty
student@okan-lab:~/lab$ rm projects/beta
rm: cannot remove 'projects/beta': Is a directory
student@okan-lab:~/lab$ cp -r projects/beta beta_copy
student@okan-lab:~/lab$ tree beta_copy
...
3 directories, 3 files
student@okan-lab:~/lab$ rm -rv beta_copy
removed 'beta_copy/src/main.c'
removed directory 'beta_copy/src'
removed 'beta_copy/README.md'
removed 'beta_copy/docs/design.txt'
removed directory 'beta_copy/docs'
removed directory 'beta_copy'
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
```

- `rm junk1.txt` hiçbir şey yazmaz, hiçbir şey sormaz.
- `rm -i` sorar (“remove regular empty file?”, yani “boş normal dosya silinsin mi?”); `rm -v` neyi sildiğini bildirir.
- `ls junk*` başarısız olur, çünkü artık hiçbir ad kalıpla eşleşmez; bu durumda kabuk kalıbı değiştirmeden geçirir (Kısım 5).
- `rmdir` boş olmayan bir dizini reddeder; `-r` olmadan `rm` her dizini reddeder.
- `rm -rv` işin sırasını gösterir: her dizin önce boşaltılır, çünkü çekirdeğin `rmdir()` çağrısı yalnızca boş dizinleri siler.

Uyarı: rm kalıcıdır

Komut satırında **geri dönüşüm kutusu yoktur**. `rm`, `unlinkat()` çağırır; bağlantı sayısı 0'a indiğinde (ve dosyayı açık tutan bir program yoksa) bloklar boş olarak işaretlenir ve kısa sürede yeniden kullanılır. Masaüstündeki “Çöp Kutusu” çekirdeğin değil, dosya yöneticisinin bir özelliğidir.

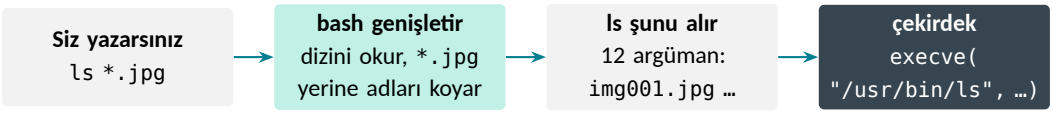
- Kalıplı bir `rm`'den önce *aynı* kalıpla `ls` çalıştırın; nerede olduğunuzu `pwd` ile denetleyin.
- `rm -r` bütün bir ağacı siler. Felaket için yanlış yerde bir boşluk yeter: `rm -r ~/lab /old`, `~/lab`'ı siler *ve* `/old`'u da dener.
- `rm -f` (force, zorla) hiç sormaz ve hataları gizler. Her karakterini anlamadığınız sürece `rm -rf` komutunu asla `sudo` ile (11. hafta) çalıştırmayın.

`~/lab` klasörünü bozarsanız kurulum betiğini yeniden çalıştırmanız yeter. Gerçek bir sistemde tek “geri al” yedeklerdir.

5 Joker Karakterler ve Süslü Parantez Genişletmesi

5.1 * karakterini kim genişletir? Kabuk

`ls *.jpg` yazdığınızda `ls` programı yıldızı hiç görmez. Komutu başlatmadan *önce* bash geçerli dizini okur, kalıpla eşleşen bütün adları bulur, onları sıralar ve komut satırına ayrı argümanlar olarak yerleştirir. Buna **glob genişletme** (globbing, pathname expansion) denir. Ardından `fork()` ve `exec()`, `ls`'yi hazır listeyle başlatır (3. hafta).



Şekil 2: Joker karakterleri komut değil, kabuk genişletir.

Üç sonucu vardır:

1. `echo` bir komutun tam olarak neyi alacağını gösterir; bu yüzden bir kalıbı denemek için en uygun araçtır.
2. Her komut joker karakterleri "destekler", çünkü hiçbir komut onları görmez.
3. Hiçbir şey eşleşmezse bash kalıbı değiştirmeden bırakır; komut da adı tam olarak `junk*` olan bir dosya arar ve böyle bir dosyanın olmadığını bildirir.

5.2 Kalıplar

Kalıp	Eşleşir	Örnek	`photos/` içindeki sonuç
`*`	herhangi bir dizgi, boş dizgi de	`*.jpg`	12 ad, img001–img012
`?`	tam olarak bir karakter	`img00?.jpg`	9 ad, img001–img009
`[0-2]`	bir aralıktan bir karakter	`img01[0-2].jpg`	img010, img011, img012
`[135]`	listelenen karakterlerden biri	`img00[135].jpg`	img001, img003, img005
`[!0]`	kümede *olmayan* bir karakter	`img0[!0]*`	img010 … img013

İki kural: Linux adları büyük/küçük harfe *duyarlıdır* (`*.jpg`, `IMG_014.JPG` ile eşleşmez) ve `*` baştaki noktayla eşleşmez: gizli dosyalar, siz noktayı yazmadıkça dışarıda kalır. `demo_wildcards.sh` betiği, 15 boş "fotoğraf" içeren `~/lab/photos` klasöründe çalışır:

```
student@okan-lab:~/lab$ cd photos
student@okan-lab:~/lab/photos$ ls
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg  img013.png
student@okan-lab:~/lab/photos$ echo *.png
img013.png
student@okan-lab:~/lab/photos$ ls img00?.jpg
img001.jpg  img003.jpg  img005.jpg  img007.jpg  img009.jpg
img002.jpg  img004.jpg  img006.jpg  img008.jpg
student@okan-lab:~/lab/photos$ ls img01[0-2].jpg
img010.jpg  img011.jpg  img012.jpg
student@okan-lab:~/lab/photos$ ls *.[jJ][pP]*
```

```
IMG_014.JPG   img002.jpg  img005.jpg  img008.jpg  img011.jpg
holiday.jpeg  img003.jpg  img006.jpg  img009.jpg  img012.jpg
img001.jpg    img004.jpg  img007.jpg  img010.jpg
```

Bir terminalde `ls` adları sütunlar hâlinde, her sütunda yukarıdan aşağıya sıralı olarak yazar. `*.[jJ][pP]*`, uzantısı j/J ve ardından p/P ile başlayan her adla eşleşir: on iki `.jpg`, `holiday.jpeg` ve `IMG_014.JPG`.

5.3 Eşleşme olmaması, tırnaklar ve gizli dosyalar

```
student@okan-lab:~/lab/photos$ echo *.gif
*.gif
student@okan-lab:~/lab/photos$ echo "*.jpg"
*.jpg
```

```
student@okan-lab:~/lab/photos$ cd ../docs
student@okan-lab:~/lab/docs$ echo *
final report.txt linux_quotes.txt notes_01.txt notes_02.txt notes_03.txt syllabus.txt
student@okan-lab:~/lab/docs$ echo .*
.settings
```

`.gif` dosyası olmadığı için `*.gif` olduğu gibi kalır. Tırnaklar genişletmeyi kapatır. `docs/` içinde `*`, `.settings` dışındaki bütün adları verir; `.*` onu bulur. (`final report.txt`'nin içinde boşluk bulunan *tek* bir argüman olduğuna dikkat edin; `echo` argümanları yalnızca aralarına boşluk koyarak yazar.)

5.4 Süslü parantez genişletmesi

Süslü parantezler bash'in ikinci ve farklı bir genişletmesidir (brace expansion). Diske hiç bakmazlar: metin *üretirler*. `{a,b,c}` listesi seçeneklerin her birini üretir; `{1..5}` aralığı bir dizi üretir; `{01..03}` baştaki sıfırları korur. Aynı sözcükteki birden çok süslü parantez çarpılır:

```
student@okan-lab:~/lab/photos$ echo {1..5}
1 2 3 4 5
student@okan-lab:~/lab/photos$ echo img{01..03}.jpg
img01.jpg img02.jpg img03.jpg
student@okan-lab:~/lab/photos$ echo report_{draft,final}.txt
report_draft.txt report_final.txt
student@okan-lab:~/lab/photos$ mkdir -p ../trip/{day1,day2,day3}/{photos,notes}
student@okan-lab:~/lab/photos$ tree ../trip
../trip
├── day1
│   ├── notes
│   └── photos
├── day2
│   ├── notes
│   └── photos
└── day3
    ├── notes
    └── photos

10 directories, 0 files
```

Çözümlü örnek: süslü parantezlerin oluşturduğu dizinleri saymak

`mkdir -p ../trip/{day1,day2,day3}/{photos,notes}` komutu $3 \times 2 = 6$ yol üretir. `trip` ve `-p`'nin oluşturduğu 3 gün diziniyle birlikte ağaçta

$$1 + 3 + 3 \times 2 = 10 \text{ dizin}$$

vardır; `tree`'nin bildirdiği tam olarak budur. Genel olarak $n_1, n_2, \ldots$ seçenekli süslü parantezler $n_1 \cdot n_2 \cdots$ sözcük üretir.

Kısacası: joker karakterler var olan adları *bulur* (sıralı; eşleşme yoksa değişmeden kalır), süslü parantezler yeni adlar *üretir* (yazıldığı sırayla, dosyalar var olsun ya da olmasın). İkisini de bash komut başlamadan önce yapar ve ikisi de tırnaklarla kapatılır. Virgülden sonraki bir boşluk (`{a, b}`) süslü parantez genişletmesini bozar.

6 Boşluklu Adlar: Tırnaklama

Kabuk, komut başlamadan *önce* komut satırını boşluklardan böler. Bu yüzden içinde boşluk bulunan bir ad, onu korumazsanız (tırnaklama, quoting) iki argümana dönüşür (`demo_quoting.sh`):

```
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ cat docs/final report.txt
cat: docs/final: No such file or directory
cat: report.txt: No such file or directory
student@okan-lab:~/lab$ cat "docs/final report.txt"
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat 'docs/final report.txt'
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat docs/final\ report.txt
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ mkdir my photos
mkdir: cannot create directory ‘photos’: File exists
student@okan-lab:~/lab$ mkdir "my photos"
student@okan-lab:~/lab$ ls -d my*
 my  'my photos'
```

- `cat`, `docs/final` ve `report.txt` adlarını aldı: iki hata.
- Çift tırnak `"…"`, tek tırnak `'…'` ya da boşluktan önce bir ters eğik çizgi (`final\ report.txt`) tek argüman verir. `"` ile `'` arasındaki fark 14. haftada önemli olacak (değişkenler yalnızca çift tırnak içinde genişletilir). Tab tamamlama ters eğik çizgiyi sizin için ekler.
- `mkdir my photos`, `my`'ı oluşturdu ve zaten var olan `photos`'ta başarısız oldu.
- `ls` boşluklu adları tırnak içinde yazar (`'my photos'`); böylece onları kopyalayıp bir komuta geri yapıştırabilirsiniz.

Öneri: kendi oluşturduğunuz adlarda boşluk yerine `_` ya da `-` kullanın. (Temizlik: `rmdir my "my photos"`.)

7 Dosyaların İçine Bakmak

7.1 `cat`, `head`, `tail`

`cat` (*concatenate*, art arda eklemek) dosyaların tamamını yazar; birden çok dosyayı birbiri ardına yazar ve `-n` satırları numaralar. `head -n N` ve `tail -n N` ilk ve son N satırı yazar (varsayılan 10) (`demo_view.sh`):

```
student@okan-lab:~/lab$ cat docs/linux_quotes.txt
Talk is cheap. Show me the code.
Given enough eyeballs, all bugs are shallow.
Most good programmers do programming not because they expect to get paid.
Theory and practice sometimes clash. And when that happens, theory loses.
Intelligence is the ability to avoid doing work, yet getting the work done.
student@okan-lab:~/lab$ cat -n docs/notes_01.txt docs/notes_02.txt
     1  Meeting notes 1
     2  Meeting notes 2
student@okan-lab:~/lab$ head -n 3 data/students.csv
id,name,department,midterm,final
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ tail -n 3 logs/system.log
2026-10-12 14:27:00 INFO  user login
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
```

7.2 `tail -f` ve `less`

`tail -f` (*follow*, izle) dosyayı açık tutar ve başka programlar yeni satırlar ekledikçe onları yazar — bir günlük (log) dosyasını izlemenin standart yolu budur. İki terminal açın: birincide `tail -n 2 -f logs/system.log` çalıştırın, ikincide bir satır ekleyin:

```
$ echo "2026-10-12 15:00:00 WARN  disk almost full" >> logs/system.log
```

Yeni satır hemen birinci terminalde belirir; `tail`'i Ctrl+C ile durdurun (^C olarak görünür). Betiğimizde ikinci terminalin rolünü bir arka plan işi oynar:

```
student@okan-lab:~/lab$ tail -n 2 -f logs/system.log
2026-10-12 14:38:00 INFO  backup finished
2026-10-12 14:49:00 INFO  network link up
2026-10-12 15:00:00 WARN  disk almost full
^C
```

Uzun dosyalarda `cat` çok hızlı kayar. `less dosya` bir seferde bir ekran gösterir ve yalnızca görüntülediği kısmı okuduğu için çok büyük dosyaları bile anında açar. 9. haftadaki `man` sayfaları da `less` ile gösterilir:

Tuş	İşlem	Tuş	İşlem
`Space` / `b`	sonraki / önceki sayfa	`/sözcük`	ileri doğru ara
`↓` `↑`	bir satır	`n` `N`	sonraki / önceki eşleşme
`g` `G`	dosyanın başı / sonu	`q`	çık

7.3 `wc` ve `file`

```
student@okan-lab:~/lab$ wc docs/linux_quotes.txt
  5  50 302 docs/linux_quotes.txt
student@okan-lab:~/lab$ wc -l data/*.csv data/*.txt
  41 data/students.csv
  30 data/cities.txt
  20 data/numbers.txt
  91 total
student@okan-lab:~/lab$ file README.txt src/hello.c data/students.csv photos/img001.jpg
    projects /usr/bin/ls
README.txt:        ASCII text
src/hello.c:       C source, ASCII text
data/students.csv: CSV ASCII text
photos/img001.jpg: empty
projects:          directory
/usr/bin/ls:       ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
     interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1] =
    daa5130f2a41f7fbc8662048f3294f3d439ca7ff, for GNU/Linux 3.2.0, stripped
```

wc (*word count*, sözcük sayımı) satır, sözcük ve bayt sayılarını yazar: `linux_quotes.txt` 5 satır, 50 sözcük, 302 bayttır. `-l`, `-w`, `-c` tek bir sayımı seçer. `students.csv` 41 satırdır: 1 başlık satırı $+40$ öğrenci.

`file` adı yok sayar ve içeriği, özellikle ilk baytları (*sihirli sayı*, magic number; 7. hafta) inceler. Bizim "fotoğraflarımız" `.jpg` adlı boş dosyalardır; bu yüzden `file` `empty` (boş) der. Gerçek bir JPEG `FF D8 FF` baytlarıyla başlar. `/usr/bin/ls` bir ELF yürütülebilir dosyasıdır (uzun satır alt satıra kayar).

7.4 `stat`: düğümü okumak

`stat`, 7. haftanın düğümünü alan alan yazdırır (`demo_stat_du_df.sh`):

```
student@okan-lab:~/lab$ stat docs/syllabus.txt
  File: docs/syllabus.txt
  Size: 578             Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 804123      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1010/ student)   Gid: ( 1012/ student)
Access: 2026-10-12 08:47:57.251900492 +0000
Modify: 2026-10-12 08:47:57.254669759 +0000
Change: 2026-10-12 08:47:57.254669759 +0000
 Birth: 2026-10-12 08:47:57.251900492 +0000
student@okan-lab:~/lab$ stat -c "%n: %s bytes, inode %i, %h link(s)" docs/*.txt
docs/final report.txt: 53 bytes, inode 804125, 1 link(s)
docs/linux_quotes.txt: 302 bytes, inode 804124, 1 link(s)
docs/notes_01.txt: 16 bytes, inode 804127, 1 link(s)
docs/notes_02.txt: 16 bytes, inode 804128, 1 link(s)
docs/notes_03.txt: 16 bytes, inode 804129, 1 link(s)
docs/syllabus.txt: 578 bytes, inode 804123, 1 link(s)
```

- `Size: 578` bayt veri; `Blocks: 8` ise 512 baytlık birimleri sayar: $8 \times 512 = 4096$ B, yani bir adet 4 KiB'lık dosya sistemi bloğu (`IO Block`). 578 baytlık bir dosya bile bütün bir bloğu kaplar.
- `Inode` ve `Links`: düğüm numarası ve bağlantı sayısı.
- `Access: (0664/-rw-rw-r--)`: sekizli (octal) ve sembolik biçimde izinler — gelecek haftanın konusu. `Uid`/`Gid`: sahip ve grup.
- `stat -c` yalnızca seçilen alanları yazar: `%n` ad, `%s` boyut, `%i` düğüm, `%h` sabit bağlantı sayısı, `%y`

mtime.

7.5 du ve df: ne kadar yer?

```
student@okan-lab:~/lab$ du -sh docs data logs photos
32K     docs
16K     data
36K     logs
4.0K    photos
student@okan-lab:~/lab$ du -h projects
8.0K    projects/beta/src
8.0K    projects/beta/docs
24K     projects/beta
8.0K    projects/alpha/src
8.0K    projects/alpha/docs
24K     projects/alpha
52K     projects
student@okan-lab:~/lab$ df -h ~
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G   13G   30G  29% /
```

du (*disk usage*, disk kullanımı) dosyaların ve dizinlerin kullandığı blokları toplar; `-s` her argüman için bir toplam yazar, `-h` K, M, G birimlerini kullanır. `df` (*disk free*, boş disk) dizin başına değil, *dosya sistemi* başına (bağlanmış her disk için) bildirir.

Çözümlü örnek: du'yu elle denetlemek

Her dizinin kendisi bir adet 4 KiB'lık blok kullanır. `logs/`, `access.log` ($23\,604$ B) ve `system.log` ($5\,207$ B) dosyalarını tutar. Bloklar tam olarak ayrılır:

$$\left\lceil \tfrac{23\,604}{4096} \right\rceil = 6 \text{ blok} = 24 \text{ KiB}, \qquad \left\lceil \tfrac{5\,207}{4096} \right\rceil = 2 \text{ blok} = 8 \text{ KiB},$$

dolayısıyla `logs`, du'nun bildirdiği gibi $4 + 24 + 8 = 36$ KiB kullanır. `photos/` yalnızca $4{,}0$ K kullanır: 15 dosyası boştur ve hiç veri bloğu gerektirmez. `projects/` ise $4 + 2 \times 24 = 52$ K'dir.

`df` çıktısında `Use%`, yukarı yuvarlanmış $\text{Used}/(\text{Used} + \text{Avail})$ değeridir: $12/(12 + 30) \approx 0{,}286$, yani %29 olarak gösterilir. `Size`, Used + Avail toplamından büyüktür, çünkü diskin bir kısmı ayrılmıştır ve sıradan kullanıcılara açık değildir (normal bir ext4 diskte root için %5; sanal test makinemizde çok daha fazla).

8 Bağlantılara Yeniden Bakış

7. haftada sabit (hard) ve sembolik (symbolic) bağlantılar oluşturmuştuk. Şimdi onları `~/lab/docs` içinde doğrudan bir kopyayla karşılaştırıyoruz (`demo_links.sh`):

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ ln syllabus.txt plan.txt
student@okan-lab:~/lab/docs$ cp syllabus.txt copy.txt
student@okan-lab:~/lab/docs$ ln -s syllabus.txt shortcut.txt
student@okan-lab:~/lab/docs$ ls -li syllabus.txt plan.txt copy.txt shortcut.txt
804702 -rw-rw-r-- 1 student student 578 Oct 12 08:47 copy.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
804123 -rw-rw-r-- 2 student student 578 Oct 12 08:47 syllabus.txt
```

```
student@okan-lab:~/lab/docs$ echo "Week 16  Party" >> plan.txt
student@okan-lab:~/lab/docs$ tail -n 1 syllabus.txt copy.txt
==> syllabus.txt <==
Week 16  Party

==> copy.txt <==
Week 15 Final exam
student@okan-lab:~/lab/docs$ rm syllabus.txt
student@okan-lab:~/lab/docs$ ls -li plan.txt shortcut.txt
804123 -rw-rw-r-- 1 student student 593 Oct 12 08:47 plan.txt
804703 lrwxrwxrwx 1 student student  12 Oct 12 08:47 shortcut.txt -> syllabus.txt
student@okan-lab:~/lab/docs$ cat shortcut.txt
cat: shortcut.txt: No such file or directory
```

- `syllabus.txt` ve `plan.txt`: bağlantı sayısı 2 olan *aynı* düğüm (804123) — tek dosya, iki ad (Şekil 1).
- `copy.txt`'nin kendi düğümü vardır: aynı baytları taşıyan ayrı bir dosya.
- `shortcut.txt`'nin türü `l`'dir, kendi düğümü vardır ve boyutu 12'dir; bu, sakladığı `syllabus.txt` dizgisinin uzunluğudur.
- `plan.txt` üzerinden eklenen satır `syllabus.txt`'nin sonunda görünür, `copy.txt`'de görünmez. Boyut 578'den $593 = 578 + 15$ bayta çıkar (`Week 16  Party` 14 karakter, artı bir satır sonu).
- `rm syllabus.txt`'den sonra veriye hâlâ `plan.txt` adıyla ulaşılabilir (bağlantı sayısı yeniden 1); sembolik bağlantı ise *askıda kalır* (dangling link): sakladığı yol artık yoktur.

Sembolik bağlantılar dizinleri gösterebilir; dizinlere sabit bağlantı ise yasaktır, çünkü dizin çizgesinde döngüler oluşturabilirler (7. hafta, genel çizge dizinler):

```
student@okan-lab:~/lab/docs$ cd ..
student@okan-lab:~/lab$ ln -s projects/alpha/docs alpha_docs
student@okan-lab:~/lab$ ls -l alpha_docs
lrwxrwxrwx 1 student student 19 Oct 12 08:47 alpha_docs -> projects/alpha/docs
student@okan-lab:~/lab$ ls alpha_docs/
design.txt
student@okan-lab:~/lab$ ln projects alpha_hard
ln: projects: hard link not allowed for directory
student@okan-lab:~/lab$ ls -ld projects projects/alpha
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects
drwxrwxr-x 4 student student 4096 Oct 12 08:47 projects/alpha
```

Bir dizinin bağlantı sayısı $2 + (\text{alt dizin sayısı})$'dır: üst dizindeki adı, kendi `.` girdisi ve her çocuğun `..` girdisi. `projects`, `alpha` ve `beta`'yı içerir; dolayısıyla $2 + 2 = 4$.

	Sabit bağlantı `ln`	**Sembolik bağlantı `ln -s`**	**Kopya `cp`**
Düğüm	hedefle aynı	kendine ait (bir yol saklar)	kendine ait
Veri	paylaşılır	yol üzerinden ulaşılır	çoğaltılır
Hedef silinirse	veri kalır (sayı -1)	bağlantı askıda kalır	kopya etkilenmez
Dizinler	izin verilmez	izin verilir	`-r` ile
Başka dosya sistemi	olanaksız	olanaklı	olanaklı

Göreli bir sembolik bağlantı, sizin geçerli dizininize göre değil, *bağlantıyı içeren dizine* göre yorumlanır. Bu yüzden `~/lab` içinde yazılan

```
ln -s syllabus.txt docs/shortcut.txt
```

komutu `docs/syllabus.txt`'yi gösterir — sık karşılaşılan bir kafa karışıklığı kaynağı.

8.1 Uygulama: aynı işlemler C'de

`code/fileops.c` programı sistem çağrılarını doğrudan yapar; bu haftanın her komutu için bir çağrı. `CHECK` makrosu, bir çağrı başarısız olduğunda nedenini `perror()` ile yazar; tıpkı komutların hata iletilerini yazması gibi. Programın özü:

```c
CHECK(mkdir("box", 0775));                                  /* mkdir box         */
CHECK(fd = open("box/a.txt", O_WRONLY | O_CREAT, 0664)); /* touch box/a.txt   */
CHECK(write(fd, "hello\n", 6));  CHECK(close(fd));
CHECK(link("box/a.txt", "box/b.txt"));                      /* ln a.txt b.txt    */
CHECK(symlink("a.txt", "box/s.txt"));                       /* ln -s a.txt s.txt */
CHECK(rename("box/a.txt", "box/c.txt"));                    /* mv a.txt c.txt    */
CHECK(unlink("box/b.txt"));                                 /* rm b.txt          */
if (rmdir("box") == -1) perror("rmdir(\"box\")");           /* not empty: fails  */
CHECK(unlink("box/c.txt"));  CHECK(unlink("box/s.txt")); /* rm c.txt s.txt    */
CHECK(rmdir("box"));                                        /* rmdir box: now OK */
```

```
student@okan-lab:~/lab$ gcc -Wall -O2 fileops.c -o fileops
student@okan-lab:~/lab$ ./fileops
after open(O_CREAT):
  box/a.txt  inode 804677  links 1
after link() and symlink():
  box/a.txt  inode 804677  links 2
  box/b.txt  inode 804677  links 2
  box/s.txt  inode 804678  links 1
after rename() a.txt -> c.txt:
  box/c.txt  inode 804677  links 2
after unlink() b.txt:
  box/c.txt  inode 804677  links 1
rmdir("box"): Directory not empty
box removed
```

`link()`'ten sonra iki ad da 2 bağlantılı aynı düğümü gösterir; `rename()` düğümü korur; `unlink()`'ten sonra sayı yeniden 1'dir. Boş olmayan `box` üzerinde `rmdir()` `Directory not empty` (dizin boş değil) ile başarısız olur — kabukta `rmdir` komutunun yazdığı metnin aynısı, çünkü ikisi de çekirdeğin `ENOTEMPTY` hata kodundan gelir. Programı kendiniz `gcc -Wall -O2 fileops.c -o fileops` (ya da `make`) ile derleyin ve `~/lab` içinde çalıştırın (`demo_fileops.sh`).

9 Komut Başvuru Tablosu

Komut	Ne yapar	Yararlı seçenekler
`mkdir dir`	dizin oluşturur	`-p` bütün yol, varsa hata yok; `-v` ayrıntılı

Komut	Ne yapar	Yararlı seçenekler
`touch file`	boş dosya oluşturur ya da zamanlarını günceller	`-d "2026-01-15 09:30"` zaman ayarla
`cp src dst`	dosya kopyalar (yeni düğüm)	`-r` dizinler, `-i` sor, `-v` ayrıntılı, `-p` zamanları koru, `-a` arşiv
`mv src dst`	yeniden adlandırır ya da taşır (düğüm aynı)	`-i` üzerine yazmadan sor, `-v` ayrıntılı
`rm file`	adları siler — geri alma yok	`-i` sor, `-r` özyinelemeli, `-v` ayrıntılı, `-f` zorla (tehlikeli)
`rmdir dir`	boş bir dizini siler	`-p` boş üst dizinleri de
`ln target name`	sabit bağlantı	`ls -li` ile denetleyin
`ln -s target name`	sembolik bağlantı	`ls -l`, `name -> target` gösterir
`cat file…`	dosyaları yazar / art arda ekler	`-n` satırları numarala
`less file`	dosyada sayfa sayfa gezinir	Space, b, /sözcük, n, g, G, q
`head file`	ilk satırlar	`-n N`
`tail file`	son satırlar	`-n N`; `-f` izle
`wc file`	satır, sözcük, bayt	`-l -w -c`
`file name`	içerik türünü tanır	—
`stat file`	düğümü gösterir	`-c "%n %s %i %h %y"` seçilen alanlar
`du dir`	kullanılan disk alanı	`-s` özet, `-h` okunabilir birimler
`df`	dosya sistemi başına boş alan	`-h`; ev dizininiz için `df -h ~`
`tree dir`	dizin ağacı çizer	`-d` yalnızca dizinler
`echo pattern`	kabuğun neyi genişlettiğini gösterir	—

10 Sık Görülen Hatalar ve Anlamları

Aşağıdaki iletilerin hepsi bu haftanın gösterimlerinden alınmış gerçek çıktılardır. Türkçe yerel ayarlı bir sistemde aynı iletiler Türkçe görünebilir; anlamları değişmez.

İleti	Anlamı ve çözümü
`cp: -r not specified; omitting directory 'projects/alpha'`	Bir dizini kopyalamaya çalıştınız. `cp -r` kullanın.
`rm: cannot remove 'projects/beta': Is a directory`	`-r` olmadan `rm` dizinleri reddeder. `rmdir` (boşsa) ya da dikkatle `rm -r` kullanın.
`rmdir: failed to remove 'projects/beta': Directory not empty`	`rmdir` yalnızca boş dizinleri siler. Önce boşaltın ya da `ls` ile denetledikten sonra `rm -r` kullanın.
`mkdir: cannot create directory ‘trip/day1’: No such file or directory`	Bir üst dizin (`trip`) yok. `mkdir -p` kullanın.
`mkdir: cannot create directory ‘test’: File exists`	Ad (bir dosya ya da dizin tarafından) kullanılıyor. Ad bir dizinse `mkdir -p` hata vermez.
`cat: docs/final: No such file or directory` + `cat: report.txt: …`	Tek bir dosya adı için iki hata: adda boşluk var. Tırnak içine alın.
`ls: cannot access 'junk*': No such file or directory`	Kalıp hiçbir şeyle eşleşmedi, kabuk da onu değiştirmeden geçirdi. Kalıbı `echo` ile denetleyin.
`ln: projects: hard link not allowed for directory`	Sembolik bağlantı kullanın: `ln -s`.

İleti	Anlamı ve çözümü
`cat: shortcut.txt: No such file or directory` (ama `ls` onu gösteriyor)	Sembolik bağlantı var, hedefi yok: bağlantı askıda. `ls -l` ile denetleyin.
Hiç ileti yok	En tehlikeli durum: `cp` ve `mv` var olan bir hedefin üzerine yazar, `rm` de siler — tek söz etmeden. Emin değilseniz `-i` kullanın.

11 Özet

Yedi cümlede 10. hafta

1. Dizin adları düğümlere eşler; `cp` kopyalanmış veriyle yeni bir düğüm oluşturur, `mv` yalnızca adları değiştirir, `ln` bir ad ekler, `ln -s` bir yol saklar, `rm` bir adı siler ve bağlantı sayısı 0'a inince veri serbest kalır.
2. Her komut bir ya da birkaç sistem çağrısı yapan bir programdır: `mkdir()`, `renameat2()`, `linkat()`, `symlinkat()`, `unlinkat()`, `rmdir()`.
3. `mkdir -p` bütün yolları oluşturur; `touch` boş dosyalar oluşturur ve zamanları ayarlar; atime, mtime ve ctime sırasıyla erişimi, veri değişikliğini ve düğüm değişikliğini kaydeder.
4. `cp -r` ağaçları kopyalar; `-i` verilmedikçe `cp` ve `mv` sessizce üzerine yazar; `rm`'nin geri alması yoktur.
5. Kabuk, komut çalışmadan önce joker karakterleri (var olan adlarla eşleşir) ve süslü parantezleri (metin üretir) genişletir; `echo` sonucu gösterir, tırnaklar genişletmeyi kapatır.
6. Boşluklu adlar tırnak içine alınmalı ya da kaçışlanmalıdır.
7. `cat`, `less`, `head`, `tail -f`, `wc`, `file`, `stat`, `du` ve `df` dosyaları değiştirmeden inceler.

12 Sözlük

Askıda kalan bağlantı (dangling link) Hedefi artık var olmayan sembolik bağlantı.

Bağlantı sayısı (link count) Bir düğümü gösteren dizin girdilerinin (adların) sayısı; 0'a inince veri serbest kalır.

Düğüm (inode) Bir dosyanın adı dışındaki her şeyini (tür, boyut, sahip, izinler, zamanlar, blok işaretçileri) tutan yapı.

Glob genişletme (globbing) Kabuğun `*`, `?`, `[ ]` kalıplarını eşleşen dosya adlarıyla değiştirmesi.

Joker karakter (wildcard) Bir ad kalıbında birden çok adla eşleşen özel karakter: `*`, `?`, `[ ]`.

Özyinelemeli (recursive) Bir dizine ve altındaki her şeye aynı işlemi uygulayan (`-r`).

Sabit bağlantı (hard link) Var olan bir düğüm için ek bir dizin girdisi.

Sembolik bağlantı (symbolic link) İçeriği başka bir dosyanın yolu olan dosya.

Süslü parantez genişletmesi (brace expansion) Kabuğun `{a,b}` ve `{1..5}` biçimlerinden metin üretmesi.

Tırnaklama (quoting) Boşluk ve özel karakterleri kabuktan korumak için `"…"`, `'…'` ya da `\` kullanmak.

Zaman damgaları (timestamps) Düğümde tutulan atime (erişim), mtime (veri değişikliği), ctime (düğüm değişikliği) ve varsa Birth (oluşturulma).

13 Kendiniz Deneyin

Kısa deneyler

1. Bir dosya üzerinde `stat` çalıştırın, dosyayı `cat` ile okuyun ve `stat`'ı yeniden çalıştırın. atime değişti mi? İkinci kez neden değişmeyebilir?
2. `head -c 1000000 /dev/zero > big` ile 1 MB'lık bir dosya yapın, sonra `ls -l big` ile `du -h big` çıktılarını karşılaştırın.
3. `strace -e trace =openat,read,write cp docs/syllabus.txt /tmp/x.txt` çalıştırın ve kopyalamayı yapan `read()` ve `write()` çağrılarını bulun.

14 Tekrar Soruları

Kavramsal

1. 4 GB'lık bir dosyanın `mv` ile taşınması aynı disk içinde neden anında biter de bir USB belleğe yavaş olur?
2. `rm` ile `rmdir` arasındaki fark nedir? `rm -r` bir dizini silmeden önce neden onu boşaltmak zorundadır?
3. `*.txt` kalıbını kim genişletir: kabuk mu, komut mu? Bunu kanıtlayan bir deney verin.
4. Joker karakterlerle süslü parantez genişletmesi arasındaki farkı birer örnekle açıklayın.
5. ctime nedir ve neden oluşturulma zamanı *değildir*?
6. Özgün ad silindikten sonra sabit bağlantıyı, sembolik bağlantıyı ve kopyayı karşılaştırın.

Sayısal

7. `mkdir -p site/{css,js,img}/{v1,v2}`, `site` dahil kaç dizin oluşturur?
8. Bir dosya $10\,000$ bayttır; blok boyutu 4096 B'dir. Dosya kaç blok kullanır ve `stat` `Blocks` olarak ne gösterir?
9. Bir dizinde 5 alt dizin ve 20 dosya var. Bağlantı sayısı kaçtır?
10. `~/lab/photos` içinde (taze laboratuvar) `img0?[02468].*` kaç adla eşleşir?
11. `df` Used $= 45$ G ve Avail $= 55$ G gösteriyor. `Use%` kaçtır?

Sayısal soruların yanıtları

7. $1 + 3 + 3 \times 2 = 10$ dizin.

8. $\lceil 10\,000/4096 \rceil = 3$ adet 4 KiB'lık blok $= 12\,288$ B; `stat` 512 baytlık birimleri sayar: $3 \times 8 = 24$.
9. $2 + 5 = 7$; dosyalar bir dizinin bağlantı sayısını artırmaz.
10. Dördüncü karakter 0 ya da 1, beşinci karakter çift olmalıdır: img002, 004, 006, 008 (dördüncü karakter 0) ve img010, 012 (dördüncü karakter 1): 6 ad.
11. $45/(45 + 55) = 0{,}45$, yani %45.

15 Laboratuvar Föyü (notlu, 100 puan)

Yönerge. Taze bir laboratuvarla başlayın: `bash ~/mcmp117_lab_setup.sh && cd ~/lab`. Alıştırmaları *sırayla* yapın (sonrakiler öncekilerin sonuçlarını kullanır). Her alıştırma için kullandığınız komut(lar)ı ve çıktının önemli satırlarını yazın; bir soru "neden" diye soruyorsa tek cümleyle yanıtlayın. Kalıplı her `mv` ya da `rm`'den önce aynı kalıpla `ls` çalıştırın.

A bölümü --- Temel (33 puan)

1. **(4)** *Tek* komutla iç içe `practice/a/b/c` dizinlerini oluşturun. Sonucu `tree` ile gösterin.
2. **(4)** *Tek* komutla `practice` içinde üç boş dosya oluşturun: `one.txt`, `two.txt`, `three.txt`. `practice`'i listeleyin.
3. **(5)** `docs/syllabus.txt`'yi `backup/` içine `syllabus_backup.txt` adıyla kopyalayın; `ls -l backup` ile doğrulayın.
4. **(5)** Onu `syllabus_v1.txt` olarak yeniden adlandırın. Öncesinde ve sonrasında `ls -i` ile düğüm numarasının değişmediğini gösterin ve nedenini açıklayın.
5. **(5)** `docs/syllabus.txt`'nin ilk 4 ve son 2 satırını görüntüleyin.
6. **(5)** `docs/linux_quotes.txt` kaç satır, sözcük ve bayttır? `data/students.csv` içinde kaç *öğrenci* vardır?
7. **(5)** `docs/final report.txt`'yi iki farklı tırnaklama yoluyla görüntüleyin.

B bölümü --- Orta (39 puan)

8. **(6)** Dizin değiştirmeden `img001.jpg`–`img009.jpg` fotoğraflarını `?` ile, sonra `img010.jpg`–`img012.jpg` fotoğraflarını `[ ]` ile listeleyin; ardından hangi PNG dosyalarının bulunduğunu `echo` ile gösterin.
9. **(8)** `photos/` klasörünü düzenleyin: `jpg`, `png` ve `other` alt dizinlerini oluşturun, bütün `.jpg` dosyalarını `jpg/`'ye, bütün `.png` dosyalarını `png/`'ye, geri kalanları `other/`'a taşıyın. Sonucu `tree` ile gösterin. `IMG_014.JPG` ve `holiday.jpeg` neden `jpg/`'ye gitmez?
10. **(6)** `ls docs/*` komutunun hangi argümanları alacağını `echo` ile gösterin. `.settings` neden yok? Onu `ls` ile gösterin.
11. **(6)** Bütün `docs/` dizinini `backup/docs_copy` olarak yedekleyin. Gizli dosyanın da kopyalandığını gösterin ve iki dizinin disk kullanımını karşılaştırın.
12. **(6)** `README.txt`, `src/hello.c`, `photos/jpg/img001.jpg` ve `logs` üzerinde `file`; `data`, `logs`, `photos`, `src` üzerinde `du -sh` kullanın. Bütün fotoğraflar boş olduğu hâlde `photos` neden şimdi

16K'dir?

13. **(7)** `docs/notes_01.txt`'nin erişim ve değişiklik zamanını 1 Eylül 2026, 12:00 olarak ayarlayın. İki zamanı `stat -c` ile gösterin ve `ls -l`'nin şimdi neden yıl değil de günün saatini gösterdiğini açıklayın.

C bölümü --- Zor (28 puan)

14. **(7)** `docs/syllabus.txt`'ye `docs/syllabus_hard.txt` adında bir sabit bağlantı oluşturun. Düğüm numaralarını ve bağlantı sayılarını gösterin. `docs/syllabus.txt`'yi silin ve verinin hâlâ var olduğunu gösterin. Bağlantı sayısı şimdi kaçtır?

15. **(8)** `~/lab` içinde `projects/alpha/docs`'u gösteren `alpha_docs` adlı bir sembolik bağlantı oluşturun ve içeriğini bağlantı üzerinden listeleyin. `projects/alpha`'yı `projects/alpha_old` olarak yeniden adlandırın: bağlantıya ne olur? Onarın.

16. **(6)** *Tek* komutla, her biri `slides` ve `notes` içeren `course/week01` … `course/week14` dizinlerini oluşturun. Toplam kaç dizin oluşturuldu (`course` dahil)? Hesabınızı gösterin.

17. **(7)** Temizlik: `rmdir practice` komutunu deneyin ve sonucu açıklayın; sonra `practice` ve `course`'u tamamen silin ve `~/lab`'ı listeleyin.

	A bölümü	B bölümü	C bölümü	Toplam
Puan	33	39	28	100

16 Laboratuvar Föyü Çözümleri

Taze bir laboratuvarda çalıştırılan `code/demo_lab_solutions.sh` betiğinin gerçek çıktısı (düğüm numaraları ve zamanlar sizin makinenizde farklı olacaktır).

A bölümü

1–2. `-p` bütün yolu oluşturur; süslü parantez genişletmesi tek bir `touch`'a üç ad verir:

```
student@okan-lab:~/lab$ mkdir -p practice/a/b/c
student@okan-lab:~/lab$ tree practice
practice
└── a
    └── b
        └── c

4 directories, 0 files
student@okan-lab:~/lab$ touch practice/{one,two,three}.txt
student@okan-lab:~/lab$ ls practice
a  one.txt  three.txt  two.txt
```

3–4. Düğüm numarası `mv`'den önce ve sonra aynıdır, çünkü yeniden adlandırma yalnızca dizin girdisini değiştirir; düğüme ve veriye dokunulmaz:

```
student@okan-lab:~/lab$ cp docs/syllabus.txt backup/syllabus_backup.txt
student@okan-lab:~/lab$ ls -l backup
total 4
-rw-rw-r-- 1 student student 578 Oct 12 08:48 syllabus_backup.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_backup.txt
804718 backup/syllabus_backup.txt
```

```
student@okan-lab:~/lab$ mv backup/syllabus_backup.txt backup/syllabus_v1.txt
student@okan-lab:~/lab$ ls -i backup/syllabus_v1.txt
804718 backup/syllabus_v1.txt
```

5–7. `students.csv` 41 satırdır ve bunlardan biri başlıktır; yani $41 - 1 = 40$ öğrenci:

```
student@okan-lab:~/lab$ head -n 4 docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
student@okan-lab:~/lab$ tail -n 2 docs/syllabus.txt
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ wc docs/linux_quotes.txt
  5  50 302 docs/linux_quotes.txt
student@okan-lab:~/lab$ wc -l data/students.csv
41 data/students.csv
student@okan-lab:~/lab$ cat "docs/final report.txt"
Final report draft.
This file name contains a space.
student@okan-lab:~/lab$ cat docs/final\ report.txt
Final report draft.
This file name contains a space.
```

B bölümü

8.

```
student@okan-lab:~/lab$ ls photos/img00?.jpg
photos/img001.jpg  photos/img004.jpg  photos/img007.jpg
photos/img002.jpg  photos/img005.jpg  photos/img008.jpg
photos/img003.jpg  photos/img006.jpg  photos/img009.jpg
student@okan-lab:~/lab$ ls photos/img01[0-2].jpg
photos/img010.jpg  photos/img011.jpg  photos/img012.jpg
student@okan-lab:~/lab$ echo photos/*.png
photos/img013.png
```

9. `IMG_014.JPG`, adlar büyük/küçük harfe duyarlı olduğu için `*.jpg` ile eşleşmez; `holiday.jpeg`'in ise uzantısı farklıdır. İkisini de üçüncü `mv` taşır. (Son adım için daha kısa bir seçenek: `mv *.* other/`, çünkü üç yeni dizinin adında nokta yoktur.)

```
student@okan-lab:~/lab$ cd photos
student@okan-lab:~/lab/photos$ mkdir jpg png other
student@okan-lab:~/lab/photos$ mv *.jpg jpg/
student@okan-lab:~/lab/photos$ mv *.png png/
student@okan-lab:~/lab/photos$ mv *.JPG *.jpeg other/
student@okan-lab:~/lab/photos$ tree
.
├── jpg
│   ├── img001.jpg
│   ├── ...
│   └── img012.jpg
├── other
│   ├── IMG_014.JPG
│   └── holiday.jpeg
└── png
    └── img013.png

4 directories, 15 files
student@okan-lab:~/lab/photos$ cd ..
```

10–11. `*` nokta ile başlayan adlarla eşleşmez; bu yüzden `.settings` argümanlar arasında yoktur; `ls -a` onu gösterir. `cp -r` gizli dosyalar dahil her şeyi kopyalar (`ls -A` onları `.` ve `..` olmadan listeler); iki ağaç da 32 K kullanır:

```
student@okan-lab:~/lab$ echo docs/*
docs/final report.txt docs/linux_quotes.txt docs/notes_01.txt docs/notes_02.txt docs/notes_03.txt docs/
    syllabus.txt
student@okan-lab:~/lab$ ls -a docs
 .    .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
 ..  'final report.txt'   notes_01.txt       notes_03.txt
student@okan-lab:~/lab$ cp -r docs backup/docs_copy
student@okan-lab:~/lab$ ls -A backup/docs_copy
 .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
'final report.txt'   notes_01.txt       notes_03.txt
student@okan-lab:~/lab$ du -sh docs backup/docs_copy
32K     docs
32K     backup/docs_copy
```

12. `photos/` artık üç alt dizin içerir; her dizin bir adet 4 KiB'lık blok kullanır: $4 \times 4 = 16$ K; boş fotoğraf dosyaları ise hâlâ 0 blok kullanır:

```
student@okan-lab:~/lab$ file README.txt src/hello.c photos/jpg/img001.jpg logs
README.txt:            ASCII text
src/hello.c:           C source, ASCII text
photos/jpg/img001.jpg: empty
logs:                  directory
student@okan-lab:~/lab$ du -sh data logs photos src
16K     data
36K     logs
16K     photos
8.0K    src
```

13. 1 Eylül bugünden altı aydan daha az önce olduğu için `ls -l` günün saatini yazar:

```
student@okan-lab:~/lab$ touch -d "2026-09-01 12:00" docs/notes_01.txt
student@okan-lab:~/lab$ stat -c "%n | access %x | modify %y" docs/notes_01.txt
docs/notes_01.txt | access 2026-09-01 12:00:00.000000000 +0000 | modify 2026-09-01 12:00:00.000000000
    +0000
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student student 16 Sep  1 12:00 docs/notes_01.txt
```

C bölümü

14. Aynı düğüm, bağlantı sayısı 2; `rm`'den sonra sayı 1'dir ve veri sağlamdır:

```
student@okan-lab:~/lab$ ln docs/syllabus.txt docs/syllabus_hard.txt
student@okan-lab:~/lab$ ls -li docs/syllabus.txt docs/syllabus_hard.txt
804121 -rw-rw-r-- 2 student student 578 Oct 12 08:48 docs/syllabus.txt
804121 -rw-rw-r-- 2 student student 578 Oct 12 08:48 docs/syllabus_hard.txt
student@okan-lab:~/lab$ rm docs/syllabus.txt
student@okan-lab:~/lab$ ls -li docs/syllabus_hard.txt
804121 -rw-rw-r-- 1 student student 578 Oct 12 08:48 docs/syllabus_hard.txt
student@okan-lab:~/lab$ head -n 2 docs/syllabus_hard.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
```

15. Yeniden adlandırmadan sonra bağlantı askıda kalır (saklanan `projects/alpha/docs` yolu artık yoktur); bağlantıyı silip yeni yolla yeniden oluşturun. Tek adımlık seçenek: `ln -sfn projects/alpha_old/docs alpha_docs`.

```
student@okan-lab:~/lab$ ln -s projects/alpha/docs alpha_docs
```

```
student@okan-lab:~/lab$ ls alpha_docs/
design.txt
student@okan-lab:~/lab$ mv projects/alpha projects/alpha_old
student@okan-lab:~/lab$ ls alpha_docs/
ls: cannot access 'alpha_docs/': No such file or directory
student@okan-lab:~/lab$ rm alpha_docs
student@okan-lab:~/lab$ ln -s projects/alpha_old/docs alpha_docs
student@okan-lab:~/lab$ ls -l alpha_docs
lrwxrwxrwx 1 student student 23 Oct 12 08:48 alpha_docs -> projects/alpha_old/docs
```

16. 1 (`course`) $+14$ (haftalar) $+14 \times 2$ (`slides`, `notes`) $= 43$ dizin:

```
student@okan-lab:~/lab$ mkdir -p course/week{01..14}/{slides,notes}
student@okan-lab:~/lab$ ls course
week01  week03  week05  week07  week09  week11  week13
week02  week04  week06  week08  week10  week12  week14
student@okan-lab:~/lab$ ls course/week10
notes  slides
```

17. `practice` boş olmadığı için `rmdir` reddeder; `rm -r` iki ağacı da siler (adları önce `ls` ile denetleyin):

```
student@okan-lab:~/lab$ rmdir practice
rmdir: failed to remove 'practice': Directory not empty
student@okan-lab:~/lab$ rm -r practice course
student@okan-lab:~/lab$ ls
README.txt  backup  docs  photos    scripts
alpha_docs  data    logs  projects  src
```

17 İleriye Bakış: 11. Hafta

Bugün `ls -l`'nin ilk sütununu, `-rw-rw-r--`'yi, ve `stat`'ın `Access: (0664/-rw-rw-r--)` satırını görmezden geldik. Gelecek hafta, **İzinler, Sahiplik ve Kullanıcılar**, bunları açıklıyor: kullanıcılar ve gruplar (`id`, `/etc/passwd`), dosyalar ve dizinler için okuma/yazma/çalıştırma bitleri, sembolik ve sekizli biçimde `chmod` (örneğin $r = 4$, $w = 2$, $x = 1$ ile $754 = 7|5|4$), `umask`, `chown` ve `sudo`. Bunların hepsi bugün kullandığımız aynı düğümde saklanır.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 13 (File-System Interface) ve Bölüm 14 (File-System Implementation).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Bölüm 39, "Interlude: Files and Directories" (sabit ve sembolik bağlantılar, `rename`, `unlink`, `stat`): `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- W. Shotts, *The Linux Command Line*, Bölüm 4 (Manipulating Files and Directories) ve Bölüm 7 (Seeing the World as the Shell Sees It); ücretsiz: `https://linuxcommand.org/tlcl.php`.